

Repetition

13 May 2026 10:53

```
print(1)
print(2)
print(3)
print(4)
print(5)
print(6)
```

```
for i in range(1, 100):
    print(i)
```

start → stop-1

This kind of repetition is also called iteration. Repetition of a set of statements in a program is made possible using looping constructs.

Syntax of the for loop

```
for <control_variable> in range(start, stop):
    <statements inside body of the for loop>
```

For example:

```
n = int(input('Enter number of terms: '))
for i in range(1, n):
    print(i, end=' ')
```

Output:

```
Enter number of terms: 31
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
```

Syntax of the for loop with skip value or jump value

```
for <control_variable> in range(start, stop, skip_value(default value is
1, if this is omitted):
    <statements inside body of the for loop>
```

For example:

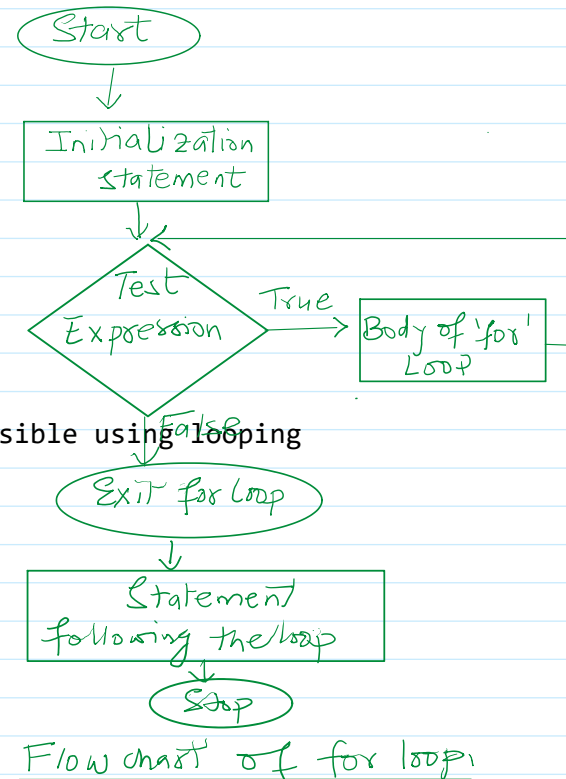
```
n = int(input('Enter number of terms: '))
for i in range(1, n, 2):
    print(i, end=' ')
```

Output:

```
Enter number of terms: 31
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29
```

Another example:

```
start = int(input('Enter start value: '))
end = int(input('Enter end value: '))
sv = int(input('Enter skip or Jump: '))
```



```
for i in range(start, end+1, sv):
    print(i, end=' ')
```

Output:

```
Enter start value: 10
Enter end value: 100
Enter skip or Jump: 5
10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90 95 100
```

Reverse:

```
start = int(input('Enter start value: '))
end = int(input('Enter end value: '))
sv = int(input('Enter skip or Jump: '))
for i in range(start, end+1, sv):
    print(i, end=' ')

print('\nReverse: ')
for i in range(end, start-1, -sv):
    print(i, end = ' ')
```

Output:

```
Enter start value: 10
Enter end value: 100
Enter skip or Jump: 5
10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90 95 100
Reverse:
100 95 90 85 80 75 70 65 60 55 50 45 40 35 30 25 20 15 10
```

WAP to print n terms of odd numbers divisible by 3.

Input: Enter number of terms: 5

Output: 3, 9, 15, 21, 27

Homework:

1. Write a Python program to print numbers from 1 to 50 using a for loop.
2. Write a Python program to print the first n natural numbers using:
for i in range(1, n+1)
3. Write a Python program to print all even numbers between 1 and 100 using a for loop.
4. Write a Python program to print all odd numbers between 1 and 50 using:
range(start, stop, skip_value)
5. Write a Python program to print multiplication table of a given number using a for loop.

Example:

Input: 5

Output:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

6. Write a Python program to print numbers from 100 to 1 in reverse order using:
range(start, stop, -1)
7. Write a Python program to print numbers from 10 to 100 with a jump value of 10.

Expected Output:

```
10 20 30 40 50 60 70 80 90 100
```

8. Write a Python program to print the first n numbers divisible by 5.

Example:

Input: 6

Output:

5 10 15 20 25 30

9. Write a Python program to print all numbers between two user-given values using a user-defined skip/jump value.

Example:

Input:

Start = 2

End = 20

Jump = 3

Output:

2 5 8 11 14 17 20

10. Write a Python program to print the reverse of a sequence entered by the user using a negative skip value.

Example:

Input:

Start = 1

End = 20

Jump = 2

Output:

19 17 15 13 11 9 7 5 3 1

11. Write a Python program to print the first n multiples of 7 using a for loop.

Example:

Input: 5

Output:

7 14 21 28 35

12. Write a Python program to print the first n odd numbers divisible by 3.

Example:

Input: 5

Output:

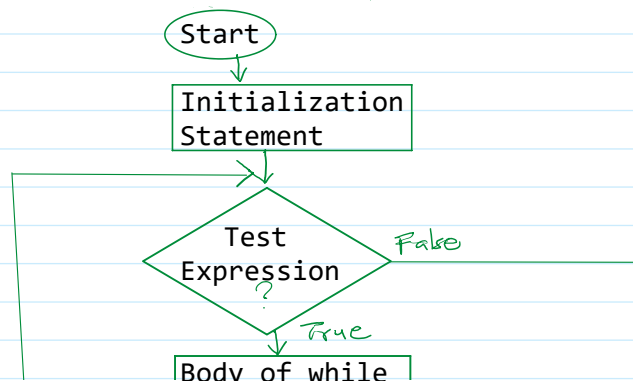
3 9 15 21 27

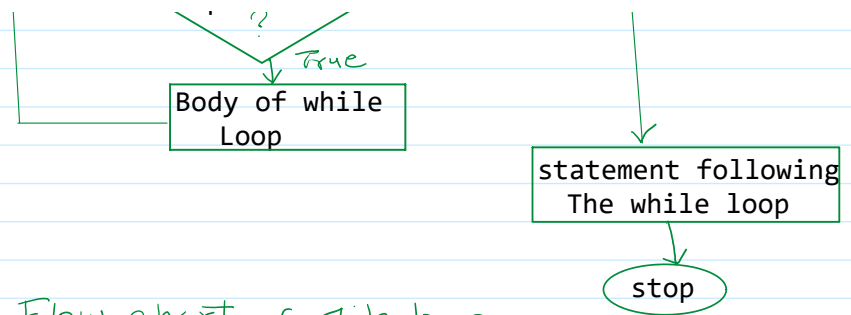
The "while" loop

While statement executes a block of code repeatedly as long as the control condition of the loop is true. The control condition of the while loop is executed before any statement inside the loop is executed. After each iteration, the control condition is tested again and the loop continues as long as the condition remains true. When this condition becomes false, the statement in the body of the loop are not executed and the control is transferred to the statement immediately following the body of the while you. If the condition of the while loop is initially false, the body is not executed even once.

The syntax:

```
while test_condition:  
    Body of while
```





Flow chart of while loop.

Program to print first n natural numbers

```

count = 1
N = int(input('Enter number of terms: '))
while count <= N:
    print(count)
    count += 1
  
```

count = 1 2 3 4 5 6
N = 5

Output:

```

Enter number of terms: 5
1 ✓
2 ✓
3 ✓
4 ✓
5 ✓
  
```

Program to find the factors of a whole number using while loop.

Factor of 12 is (1, 2, 3, 4, 6, 12)

Factor of 6 is (1, 2, 3, 6)

```

N = int(input('Enter any positive integer values greater than 1: '))
if(N <= 1):
    print('Invalid input')
else:
    factor = 2
    print(1, end=', ')
    while factor <= N//2:
        if N % factor == 0:
            print(factor, end=', ')
            factor += 1
    print(N)
  
```

N = 8
factor = 2 3 4 5

1, 2, 4, 8

output = 1, 2, 4, 8

Output:

```

Enter any positive integer values greater than 1: 8
1, 2, 4, 8
PS C:\Users\bluej\Documents\Students\ShivRay\Forgit\Coding>
Students\ShivRay\Forgit\Coding\Looping\Factors.py
Enter any positive integer values greater than 1: 12
1, 2, 3, 4, 6, 12
  
```

"break" and "continue" statement

Looping constructs allow programmers to repeat task efficiently. In certain situations, when some particular condition occur, we may want to exit from the loop. (Come out of the loop forever.) Or skip some statements of the loop before continuing further in the loop. These are requirements can be achieved by using brake and continue statements respectively. Python provides these statements as a tool to give more flexibility to the programmer to control the flow of execution of a programmer.

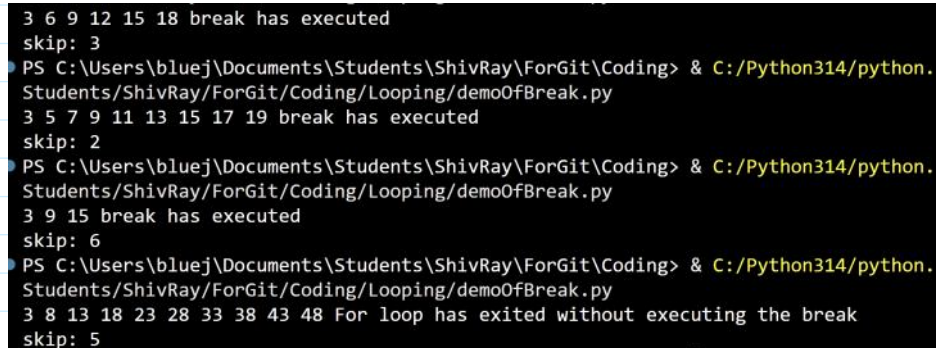
break Statement: The break statement. Alters the normal flow of execution as it terminates the current loop and resumes the execution of the statement following that loop.

For example:

```
import random as R
breakHasExecuted = False
skip = R.randint(1, 7)
for i in range(3, 50, skip):
    if i % 3 == 0 and i % 7 == 0:
        breakHasExecuted = True
        break
    print(i, end=' ')
else:
    print('For loop has exited without executing the break')
if breakHasExecuted:
    print('break has executed')

print('skip:', skip)
```

Output:



```
3 6 9 12 15 18 break has executed
skip: 3
PS C:\Users\bluej\Documents\Students\ShivRay\ForGit\Coding> & C:/Python314/python.
Students/ShivRay/ForGit/Coding/Looping/demoOfBreak.py
3 5 7 9 11 13 15 17 19 break has executed
skip: 2
PS C:\Users\bluej\Documents\Students\ShivRay\ForGit\Coding> & C:/Python314/python.
Students/ShivRay/ForGit/Coding/Looping/demoOfBreak.py
3 9 15 break has executed
skip: 6
PS C:\Users\bluej\Documents\Students\ShivRay\ForGit\Coding> & C:/Python314/python.
Students/ShivRay/ForGit/Coding/Looping/demoOfBreak.py
3 8 13 18 23 28 33 38 43 48 For loop has exited without executing the break
skip: 5
```

"continue" Statement

When a **continue** statement is encountered, the control skips the execution for the remaining statements inside the body of the loop for the current iteration and jumps to the beginning of the loop for the next iteration. If the loop's condition is still true, the loop is entered again, else the control is transferred to the statement immediately following the loop.

Program to demonstrate the use of continuous statement.

```
import random as R
breakHasExecuted = False
```

```

skip = R.randint(1, 7)
count = 0
for i in range(3, 50, skip):
    if i % 3 == 0 and i % 7 == 0:
        continue

    print(i, end=' ')
    count += 1
    if count % 10 == 0:
        print()
else:
    print('\nFor loop has exited')

print('skip:', skip)

```

Output:

```

PS C:\Users\bluej\Documents\Stude
Students/ShivRay/ForGit/Coding/Lc
3 9 15 27 33 39 45
For loop has exited
skip: 6
PS C:\Users\bluej\Documents\Stude
Students/ShivRay/ForGit/Coding/Lc
3 8 13 18 23 28 33 38 43 48
For loop has exited
skip: 5
PS C:\Users\bluej\Documents\Stude
Students/ShivRay/ForGit/Coding/Lc
3 6 9 12 15 18 24 27 30 33
36 39 45 48
For loop has exited
skip: 3
PS C:\Users\bluej\Documents\Stude

```

How to create a range?

```

n = int(input('Enter any number: '))
start = 1
while start <= n:
    # your code
    start += 1

```

Handwritten notes for a series program:

```

x = 1, sign = 1
1, -3, 5, -7, 9
while x <= N:
    print((x*2-1) * sign)
    sign = sign * (-1)
    x += 1
if x % 2 == 0:
    print(-(x*2-1), end='; ')
else:
    print((x*2-1))

```

Additional notes on the right:

- 4. - = -
- - = +
- x = 1, 2, 3
- N = 10
- sign = 1, -1, 1, -1, 1, -1, 1, -1, 1, -1

Write a program to display the first ten terms of the series:

- (a) 1, -3, 5, -7, 9, ...
- (b) 2, 5, 10, 17, ...
- (c) 0, 1, 2, 3, 6, ...
- (d) 0, 3, 8, 15, ...
- (e) 1, 12, 123, 1234, ...

Write a program to enter two numbers and check whether they are co-prime or not Sample Input: 14, 15 Sample Output: They are co-prime.

Handwritten notes for co-prime check:

a, b, c, d

0, 1, 2, 3, 6

Handwritten notes for co-prime check:

```

a = -1, b = 0, c = 1
d = a + b + c
a = b
b = c
c = d

```

Handwritten notes for co-prime check:

```

a = 1, b = 0, c = 1
b = 0 + 0 + 2
c = 1 + 0 + 2 = 3
d = 0, 1, 2, 3, 6

```

Write a program in Java to find the sum of the given series:

- (a) $S = 9 + 99 + 8 + 89 + 7 + \dots + n$ terms
- (b) $S = 1 + 1 + 2 + 3 + 5 + \dots + n$ terms
- ✓ (c) $S = 2 - 4 + 6 - 8 + \dots + n$ terms
- (d) $S = (1*2) + (2*3) + \dots + (n*(n+1))$

```
# S = 2 - 4 + 6 - 8 + ... n terms
n = int(input('Enter number of terms: '))
sign = 1
sum = 0
for i in range(1, n):
    d = i*2*sign
    if sign == 1:
        print(d, end = '')
    else:
        print(d, '+', end = '')
    sum += d
    sign = sign * (-1)
else: # this else is the part of for loop
    d = n * 2 * sign
    sum += d
    print(d, ' = ', sum)
```

[1, 2, 3, 4]

$n = 5$
 $sign = + - + - +$
 $sum = 0$
 $i = 1, 2, 3, 4$
 $d = 2, -4, 6, -8, 10$
 $sum = 2 - 4 + 6 - 8 + 10$

output:
 $2 - 4 + 6 - 8 + 10 = 6$

Nested loops

A loop may contain another loop inside it. A loop inside another loop is called nested loop.

Thinking skill: The inner loop will always complete its execution for each step of outer loop. You can compare this analogy with a clock.

Syntax:

Outer Loop

Code

Inner or nested Loop

Code

```
for i in range(3):
    print("Iteration", i, 'of outer loop')
    for j in range(5):
        print('\tj =', j)
    print('-'*50)
```

} inner or nested completes its executions for each value of outer loop variable.

Output:

Iteration 0 of outer loop

j = 0
j = 1
j = 2
j = 3
j = 4

Iteration 1 of outer loop

```
j = 0
j = 1
j = 2
j = 3
j = 4
```

Iteration 2 of outer loop

```
j = 0
j = 1
j = 2
j = 3
j = 4
```

```
i = 0
while i < 3:
    print("Iteration", i, 'of outer loop')
    j = 0
    while j < 5:
        print('\tj =', j)
        j += 1
    print('-'*50)
    i += 1
```

← nested loop, it completes its execution, for each value of 'i'.

```
i = 0
while i < 3:
    print("Iteration", i, 'of outer loop')
    for j in range(5):
        print('\tj =', j)
    print('-'*50)
    i += 1
```

```
for i in range(3):
    print("Iteration", i, 'of outer loop')
    j = 0
    while j < 5:
        print('\tj =', j)
        j += 1
    print('-'*50)
```

← nested.

Multiplication table:

```
# 5 10, 15, 20, ..., 50
# 6, 12, 18, ... 60
# ...
# ...
# n, 2n, 3n,... 10n
m = int(input('Enter the start number: '))
```



```

n = int(input('Enter the end number: '))
t = int(input('Enter number of times for each table: '))
print('-'*70)
print('\tMultiplication table from',m, 'to', n, 'upto', t, 'times')
print('-'*70)
while m <= n:
    for i in range(1, t):
        p = m * i
        print(p, end = ', ')
    else:
        print(m*t) #print the last value and end the line
    m += 1
print('-'*70)

```

Output:

```

Enter the start number: 2
Enter the end number: 10
Enter number of times for each table: 15
-----
      Multiplication table from 2 to 10 upto 15 times
-----
2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30
3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45
4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 52, 56, 60
5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75
6, 12, 18, 24, 30, 36, 42, 48, 54, 60, 66, 72, 78, 84, 90
7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84, 91, 98, 105
8, 16, 24, 32, 40, 48, 56, 64, 72, 80, 88, 96, 104, 112, 120
9, 18, 27, 36, 45, 54, 63, 72, 81, 90, 99, 108, 117, 126, 135
10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150
-----

```

WAPP to input number of rows and print the following patterns:

Input: Enter number of rows: 5

1. 1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
2. 5
5 4
5 4 3
5 4 3 2
5 4 3 2 1
3. 5 4 3 2 1
5 4 3 2
5 4 3
5 4
5
4. 1 2 3 4 5
1 2 3 4
1 2 3
1 2
1

```

1 n=int(input('Enter the number of rows: '))
2 for i in range(n): [0,1,2,3]
3     → for j in range(1,i+2): [1,2]
4         → print(j,end=' ')
5     → print()
6

```

$n = 5$
 $i = 0$
 $j = 1 \times 2$

output:

1
 $1 \ 2$
 $-$

$i = 1$
 while $i \leq n$:
 $j = 1$
 while $j \leq i$:
 $\text{print}(j, \text{end}=' ')$
 $j += 1$
 $\text{print}()$
 $i += 1$

```

thirtytwo.py > ...
1 # 5 ✓
2 # 5 4 ✓
3 # 5 4 3
4 # 5 4 3 2
5 # 5 4 3 2 1
6 n=int(input('Enter the number of rows: '))
7 for i in range(n,0,-1): [5,4,3,2,1]
8     → for j in range(n,i-1,-1): [5,4,3]
9         → print(j,end=' ')
10    → print()
11

```

$n = 5$
 $i = 5$
 $j = 4$

5 to 3 [5,4]

$n = \text{input}('Enter the number of rows: ')$
 for i in $\text{range}(n, 0, -1)$: $[5, 4, 3, 2, 1]$
 1. for j in $\text{range}(n, i-1, -1)$: $[5, 4]$
 → 1. $\text{print}(j, \text{end}=' ')$
 → 2. $\text{print}()$

output:
5
-

4, 6, 8, 10,

WAPP to input a number and check the number is prime or not.

Input: Enter any positive integer: 67

Output: Prime number

Input: Enter any positive integer: 2

Output: Prime number

Input: Enter any positive integer: 49

Output: Not a Prime number

② 3, 5, 7, 11, 13, 17, 19, 23, 29, 31

```

num = int(input("Enter any number:"))
isPrime = True
if (num > 2 and num % 2 == 0) or num == 1:
    isPrime = False
else:
    for d in range(3, num**0.5, 2):

```

Input: Enter any positive integer: 49

Output: Not a Prime number

```
num = int(input("Enter any number"))
count = 0
for d in range(1, num+1):
    if num % d == 0:
        count += 1
if count == 2:
    print("Prime number")
else:
    print("Not a prime number")
```

else:

```
for d in range(3, num+1, 2):
    if num % d == 0:
        isPrime = False
        break
if isPrime == True:
    print("Prime number")
else:
    print("Not a prime number.")
```

WAPP to enter two numbers and check whether they are co-prime or not.

Input: Enter first number: 14

Enter second number: 15

Output: They are co-prime

```
while(y > 0):
    r = x % y
    x = y
    y = r
print(hcf, x)
if x == 1:
    print("They are co-prime")
else:
    print("They aren't co-prime")
```

hcf (27, 72)

$\begin{array}{r} 27 \times 0 \\ 72 \end{array}$

$\begin{array}{r} 27 \times 2 \\ 54 \end{array}$

$\begin{array}{r} 27 \times 1 \\ 27 \end{array}$

$\begin{array}{r} 72 \times 2 \\ 144 \end{array}$

H.C.F. $\begin{array}{r} 27 \times 1 \\ 27 \end{array}$